

DSA

LAB 8:

Name: Muzammil Ahmed

Roll No:CT-24307

Q1:

Code:

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4  class Node {
5  public:
6  int data;
7  Node* left;
8  Node* right;
9  Node(int value) {
10 data = value;
11 left = nullptr;
12 right = nullptr;
13 }
14 };
15 class BinaryTree {
16 private:
17 Node* root;
18 public:
19 BinaryTree() {
20 root = nullptr;
21 }
22 void insert(int data) {
23 Node* newNode = new Node(data);
24 if (root == nullptr) {
25 root = newNode;
26 return;
27 }
28 queue<Node*> q;
29 q.push(root);
30 while (!q.empty()) {
31 Node* current = q.front();
32 q.pop();
```

```

33  if (current->left == nullptr) {
34      current->left = newNode;
35      return;
36  } else {
37      q.push(current->left);
38  }
39  if (current->right == nullptr) {
40      current->right = newNode;
41      return;
42  } else {q.push(current->right);
43  }
44  }
45  }
46  void preOrder(Node* root) {
47  if (root==NULL) {
48      return;
49  }
50      cout<<root->data<<"->";
51      preOrder(root->left);
52      preOrder(root->right);
53  }
54  void preOrderTraverse1() {
55      preOrder(root);
56      cout<<endl;
57  }
58  void postOrder(Node* root) {
59  if (root==NULL) {
60      return;
61  }

```

```

62      postOrder(root->left);
63      postOrder(root->right);
64      cout<<root->data<<"->";
65  }
66  void postOrderTraverse1() {
67      postOrder(root);
68      cout<<endl;
69  }
70  bool search(int value) {
71      if (root == nullptr) return false;
72      queue<Node*> q;
73      q.push(root);
74      while (!q.empty()) {
75          Node* current = q.front();
76          q.pop();
77          if (current->data == value) {
78              return true;
79          }
80          if (current->left != nullptr)
81              q.push(current->left);
82          if (current->right != nullptr) q.push(current->right);
83      }
84      return false;
85  }

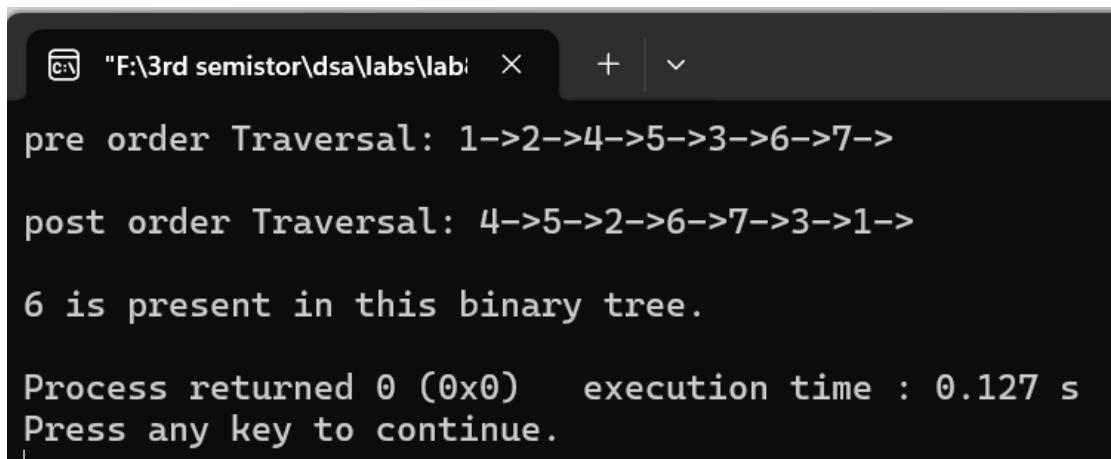
```

```

86  int main() {
87      BinaryTree tree;
88      // Insert values into the binary tree in level-order
89      tree.insert(1);
90      tree.insert(2);
91      tree.insert(3);
92      tree.insert(4);
93      tree.insert(5);
94      tree.insert(6);
95      tree.insert(7);
96      cout << "pre order Traversal: ";
97      tree.preOrderTraverse();
98      cout<<endl;
99      cout << "post order Traversal: ";
100     tree.postOrderTraverse();
101     cout<<endl;
102     int key=6;
103     if (tree.search(key))
104         cout<<key<<" is present in this binary tree."<<endl;
105     else
106         cout<<key<<" is not present in this binary tree."<<endl;
107     return 0;
108 }
109

```

Output:



```

pre order Traversal: 1->2->4->5->3->6->7->
post order Traversal: 4->5->2->6->7->3->1->
6 is present in this binary tree.
Process returned 0 (0x0)   execution time : 0.127 s
Press any key to continue.

```

Q2:

Code:

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4  class Node{
5  public:
6  int data;
7  Node* left;
8  Node* right;
9  Node(int val){
10 data=val;
11 left=right=NULL;
12 }
13 };
14 static int idx=-1;
15 Node* buildTree(vector<int> preorder){
16 idx++;
17 if(preorder[idx]==-1){
18     return NULL;
19 }
20 Node* root=new Node(preorder[idx]);
21 root->left=buildTree(preorder);
22 root->right=buildTree(preorder);
23 return root;
24 }
25 void preOrder(Node* root){
26 if(root==NULL){
27     return;
28 }
```

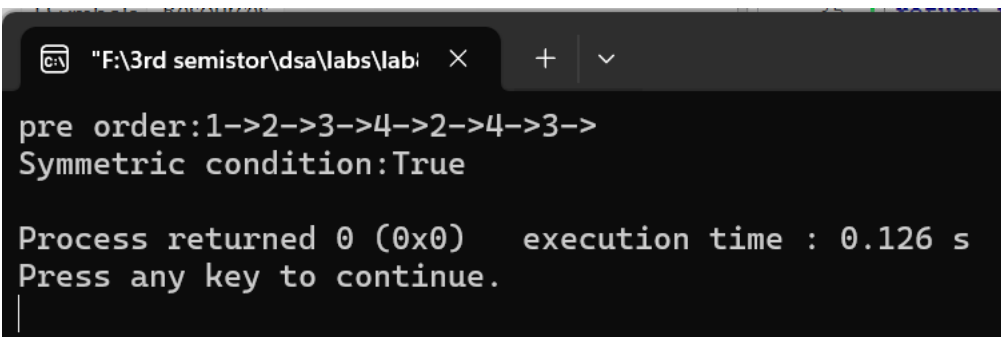
```

29     cout<<root->data<<"->";
30     preOrder(root->left);
31     preOrder(root->right);
32 }
33 bool isMirror(Node* t1, Node* t2) {
34     if(!t1 && !t2){
35         return true;
36     }
37     if(!t1 || !t2){
38         return false;
39     }
40     if(t1->data == t2->data){
41         if(isMirror(t1->left, t2->right) && isMirror(t1->right, t2->left)){
42             return true;
43         }
44     }
45     return false;
46 }
47 bool isSymmetric(Node* root){
48     if(!root)
49         return true;
50     return isMirror(root->left, root->right);
51 }

52
53 int main(){
54     vector<int> preorder = {1, 2, 3, -1, -1, 4, -1, -1, 2, 4, -1, -1, 3, -1, -1};
55     Node* root=buildTree(preorder);
56     cout<<"pre order:";
57     preOrder(root);
58     cout<<endl;
59     //checking symmetric of left and right sub tree
60     if(isSymmetric(root)==true){
61         cout<<"Symmetric condition:True"<<endl;
62     }
63     else{
64         cout<<"Symmetric condition: False"<<endl;
65     }
66     return 0;
67 }

```

Output:



```

pre order:1->2->3->4->2->4->3->
Symmetric condition:True

Process returned 0 (0x0)   execution time : 0.126 s
Press any key to continue.

```

Q3:

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  class Node{
5  public:
6  int data;
7  Node* left;
8  Node* right;
9  Node(int value) {
10 data=value;
11 left=right=nullptr;
12 }
13 };
14 // Function to check the height of the tree, or -1 if not balanced
15 int checkHeight(Node* root){
16 if(root == nullptr)
17 return 0;
18 int leftHeight =checkHeight(root->left);
19 if(leftHeight == -1)
20 return -1; // Left subtree not balanced
21 int rightHeight = checkHeight(root->right);
22 if(rightHeight==-1)
23 return -1; // Right subtree not balanced
24 if(abs(leftHeight - rightHeight) > 1)
25 return -1; // Current node not balanced
26 return 1 + max(leftHeight, rightHeight); // Return height
27 }
28
29 bool isBalanced(Node* root){
30 return checkHeight(root) != -1;
31 }
32 // Example usage
33 int main() {
34 // Balanced tree: [3,9,20,null,null,15,7]
35 Node* root1 = new Node(3);
36 root1->left = new Node(9);
37 root1->right = new Node(20);
38 root1->right->left = new Node(15);
39 root1->right->right = new Node(7);
40 cout << "Tree 1 Balanced? " << (isBalanced(root1) ? "Yes" : "No") << endl;
41 // Unbalanced tree: [1,2,null,3,null,4,null,null]
42 Node* root2 = new Node(1);
43 root2->left = new Node(2);
44 root2->left->left = new Node(3);
45 root2->left->left->left = new Node(4);
46 cout <<"Tree 2 Balanced? "<<(isBalanced(root2) ? "Yes" : "No")<<endl;
47 return 0;
48 }
```

Output:

```
"F:\3rd semistor\dsa\labs\lab: X + v
Tree 1 Balanced? Yes
Tree 2 Balanced? No

Process returned 0 (0x0)    execution time : 0.123 s
Press any key to continue.
```

Q4:

Code:

```
1  #include<iostream>
2  #include<vector>
3  #include<queue>
4  using namespace std;
5  class Node{
6  public:
7      int data;
8      Node* left;
9      Node* right;
10 Node(int val){
11     data=val;
12     left=right=NULL;
13 }
14 };
15 Node* buildTreeHelper(vector<int> preorder, vector<int> inorder, int start, int end, int& preIndex){
16     if(start>end)
17         return NULL;
18     int val=preorder[preIndex++];
19     Node* root=new Node(val);
20     if(start==end)
21         return root;
22     int inIndex = -1;
23     for(int i=start; i<=end; i++) {
24         if(inorder[i]==val) {
25             inIndex= i;
26             break;
27         }
28     }
29     root->left = buildTreeHelper(preorder, inorder, start, inIndex - 1, preIndex);
30     root->right = buildTreeHelper(preorder, inorder, inIndex + 1, end, preIndex);
31     return root;
32 }
33
34 Node* buildTree(vector<int>& preorder, vector<int>& inorder){
35     int preIndex = 0;
36     return buildTreeHelper(preorder, inorder, 0, inorder.size()-1, preIndex);
37 }
```

```

38
39 void printLevelOrder(Node* root){
40     if(!root) {
41         cout<<"[]"<<endl;
42         return;
43     }
44     queue<Node*> q;
45     q.push(root);
46     vector<string> result;
47     while(!q.empty()){
48         Node* node=q.front();
49         q.pop();
50         if(node){
51             result.push_back(to_string(node->data));
52             q.push(node->left);
53             q.push(node->right);
54         }
55         else{
56             result.push_back("null");
57         }
58     }
59     while(!result.empty() && result.back() == "null") {
60         result.pop_back();
61     }
62     cout<< "[";
63     for(size_t i = 0; i < result.size(); i++){
64         cout<<result[i];
65         if(i!=result.size()-1)
66             cout<<",";
67     }
68     cout<<"]"<<endl;
69 }
70
71 int main(){
72     vector<int> preorder={3, 9, 20, 15, 7};
73     vector<int> inorder={9, 3, 15, 20, 7};
74     Node* root=buildTree(preorder,inorder);
75     printLevelOrder(root);
76     cout<<endl;
77     return 0;
78 }

```

Output:

```

F:\3rd semistor\dsa\labs\labi  X  +  v
[3,9,20,null,null,15,7]

Process returned 0 (0x0)    execution time : 0.144 s
Press any key to continue.

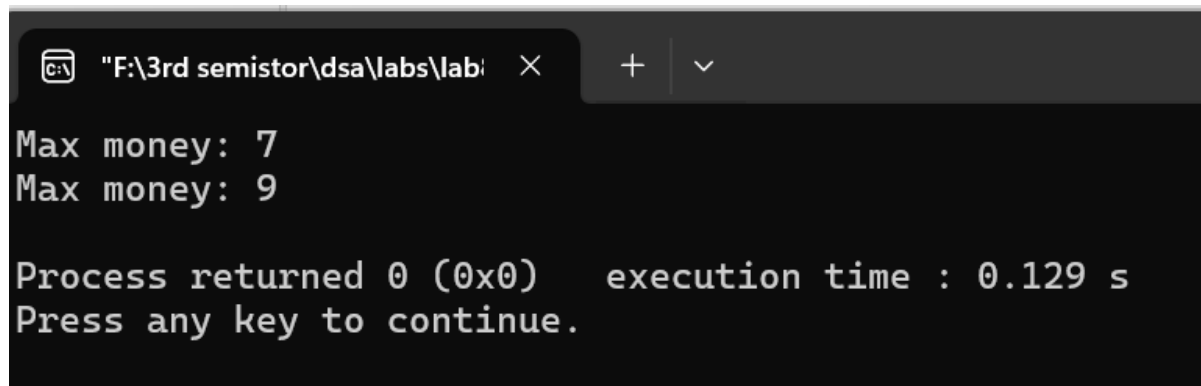
```

Q5:

Code:

```
1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  class Node{
5  public:
6  int data;
7  Node* left;
8  Node* right;
9  Node(int value) : data(value), left(nullptr), right(nullptr) {}
10 };
11 pair<int, int> robHelper(Node* node) {
12 if (node == nullptr) {
13 return {0, 0};
14 }
15 auto left = robHelper(node->left);
16 auto right = robHelper(node->right);
17 int robThis = node->data + left.second + right.second;
18 int skipThis = max(left.first, left.second) + max(right.first, right.second);
19 return {robThis, skipThis};
20 }
21 int rob(Node* root) {
22 if (!root)
23 return 0;
24 auto result = robHelper(root);
25 return max(result.first, result.second);
26 }
27
28 int main() {
29 // Example 1
30 Node* root1 = new Node(3);
31 root1->left = new Node(2);
32 root1->right = new Node(3);
33 root1->left->right = new Node(3);
34 root1->right->right = new Node(1);
35 cout<<"Max money: "<<rob(root1)<<endl;
36 // Example 2
37 Node* root2 = new Node(4);
38 root2->left = new Node(1);
39 root2->right = new Node(5);
40 root2->left->left = new Node(1);
41 root2->left->right = new Node(3);
42 root2->right->right = new Node(1);
43 cout << "Max money: " << rob(root2) << endl;
44 return 0;
45 }
```

Output:



A screenshot of a Windows command prompt window. The title bar shows the file path "F:\3rd semistor\dsa\labs\labi" with a close button. The command prompt displays the following text:

```
Max money: 7  
Max money: 9  
  
Process returned 0 (0x0)   execution time : 0.129 s  
Press any key to continue.
```